Grid
Input
JSON View
Keyboard Input
Label
Load More Button
Loading Dots
Menu
MiddleTruncate
Modal
Multi Select
Note
Pagination
Phone
Pie Gauge
Pill
Progress
Project Banner
Radio
Relative Time Card
Scroller
Search Input
Select
Separator
Sheet
Show more
Skeleton
Slider

MiddleTruncate

Truncates text in the middle, preserving the start and end of the string for maximum readability.

Examples

Covers strings that benefit from middle truncation.

Branch	feature/redesign-dashboard-navigation-with-sidebar-improvements
Preview URL	platform-web-git-feature-redesign-dashboard-navigation-phamous.vercel.app
Deployment ID	dpl_8gmXTT1yJRP8UbGfXD7A3sp4RKhW
Env var key	STRIPE_WEBHOOK_SIGNING_SECRET
Monospace no ligatures	STRIPE_WEBHOOK_SIGNING_SECRET
Commit SHA	2b0874e797d7c2a4092d0033ee0c2f0f9aef2869
File path	apps/vercel-site/app/(dashboard)/[teamSlug]/[project]/settings/page.tsx
Custom domain	api.internal.platform-observability.example.com
Model name	google/gemini-3.1-flash-image-preview
Tight width	feature/redesign-dashboard-navigation-with-sidebar-improvements
Fits as-is	sidebar.tsx

Width

600px

Animate

Hide code

```tsx
'use client';
import { Label } from '@vercel/geistcn/components';
import { Toggle } from '@vercel/geistcn/components';
import { MiddleTruncate, Slider } from '@vercel/geistcn/components';
import {
  useEffect,
  useRef,
  useState,
  type CSSProperties,
  type JSX,
} from 'react';

interface ExampleItem {
  className: string;
  label: string;
  style?: CSSProperties;
  value: string;
}

const EXAMPLES: ExampleItem[] = [
  {
    className: 'text-label-14',
    label: 'Branch',
    value: 'feature/redesign-dashboard-navigation-with-sidebar-improvements',
  },
  {
    className: 'text-copy-14',
    label: 'Preview URL',
    value:
      'platform-web-git-feature-redesign-dashboard-navigation-phamous.vercel.app',
  },
  {
    className: 'text-label-14',
    label: 'Deployment ID',
    value: 'dpl_8gmXTT1yJRP8UbGfXD7A3sp4RKhW',
  },
  {
    className: 'text-label-14 font-mono',
    label: 'Env var key',
    value: 'STRIPE_WEBHOOK_SIGNING_SECRET',
  },
  {
    className: 'text-label-14 font-mono',
    label: 'Monospace no ligatures',
    style: {
      fontFeatureSettings: '"liga" 0, "calt" 0',
      fontVariantLigatures: 'none',
    },
    value: 'STRIPE_WEBHOOK_SIGNING_SECRET',
  },
  {
    className: 'text-copy-14',
    label: 'Commit SHA',
    value: '2b0874e797d7c2a4092d0033ee0c2f0f9aef2869',
  },
  {
    className: 'text-copy-14',
    label: 'File path',
    value:
      'apps/vercel-site/app/(dashboard)/[teamSlug]/[project]/settings/page.tsx',
  },
  {
    className: 'text-copy-14',
    label: 'Custom domain',
    value: 'api.internal.platform-observability.example.com',
  },
  {
    className: 'text-label-14',
    label: 'Model name',
    value: 'google/gemini-3.1-flash-image-preview',
  },
  {
    className: 'text-label-14',
    label: 'Tight width',
    value: 'feature/redesign-dashboard-navigation-with-sidebar-improvements',
  },
  {
    className: 'text-label-14',
    label: 'Fits as-is',
    value: 'sidebar.tsx',
  },
];

const MAX_WIDTH = 600;
```

```tsx

export function Component(): JSX.Element {
  const [width, setWidth] = useState(MAX_WIDTH);
  const [isAnimating, setIsAnimating] = useState(false);
  const rafRef = useRef<number>(0);

  useEffect(() => {
    if (!isAnimating) return;

    const duration = 4000;
    let start: number | null = null;

    function step(timestamp: number): void {
      if (start === null) start = timestamp;
      const elapsed = timestamp - start;
      const progress = (elapsed % duration) / duration;
      // Ping-pong: 0→1→0→1…
      const t = progress < 0.5 ? progress * 2 : 2 - progress * 2;
      setWidth(Math.round(t * MAX_WIDTH));
      rafRef.current = requestAnimationFrame(step);
    }

    rafRef.current = requestAnimationFrame(step);
    return () => cancelAnimationFrame(rafRef.current);
  }, [isAnimating]);

  return (
    <div className="flex flex-col gap-6">
      <div className="flex flex-col gap-3">
        {EXAMPLES.map((example) => (
          <div
            className="flex items-center gap-4 rounded-md border border-gray-alpha-400 px-4 py-3"
            key={example.label}
          >
            <div className="basis-32 shrink-0 text-label-13 text-gray-700">
              {example.label}
            </div>
            <div style={{ maxWidth: width }}>
              <div className="min-w-0 basis-0 grow">
                <MiddleTruncate
                  className={example.className}
                  style={example.style}
                  value={example.value}
                />
              </div>
            </div>
          </div>
        ))}
      </div>

      <aside>
        <form className="flex gap-1 items-center">
          <Label value="Width">
            <div className="flex items-center gap-2">
              <Slider
                max={MAX_WIDTH}
                min={0}
                value={[width]}
                onValueChange={(value) => setWidth((value as number[])[0] ?? 0)}
                disabled={isAnimating}
              />

              <p className="font-mono text-copy-13 text-gray-900">{width}px</p>
            </div>
          </Label>
        </form>

        <Toggle
          checked={isAnimating}
          onChange={(): void => setIsAnimating(!isAnimating)}
        >
          Animate
        </Toggle>
      </aside>
    </div>
  );
}
```

Best Practices

When to use

- Use Middle Truncate for strings whose head and tail both carry information: file paths (`apps/…/page.tsx`), URLs, deployment IDs (`dpl_…abc123`), commit hashes, branch names with prefixes.
- For prose, descriptions, and headings, use end-truncation with `…` instead; cutting the middle of a sentence destroys meaning.
- For any truncated value the user might need verbatim, pair with a **Tooltip** showing the full string or a copy-on-click affordance.

Behavior

- Middle Truncate renders a single ellipsis glyph (`…`) rather than three periods. This keeps monospace values, including environment variable keys, IDs, hashes, and paths, from reserving three character cells for the truncation marker.
- The component listens to its container width, so layouts that change width on hover (expanding cards, animated rows) cause the truncation point to jitter. Lock the width during interaction.
- Copying truncated text yields the full original string from the underlying value, not the visible ellipsis form. Confirm this stays true if wrapping with custom `onCopy`.
- Don't wrap Middle Truncate in another `text-overflow: ellipsis` container; the two strategies fight each other and the inner ellipsis wins inconsistently.

Accessibility

- Expose the full string to assistive tech via the wrapping element's accessible name (the component already keeps the full value in the DOM for copy).
- Avoid Middle Truncate inside focusable controls without an explicit `aria-label`; the ellipsis on its own gives screen readers nothing to announce.
- Keep the visible string long enough on small viewports that the head still identifies the resource (path segment, ID prefix).